



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

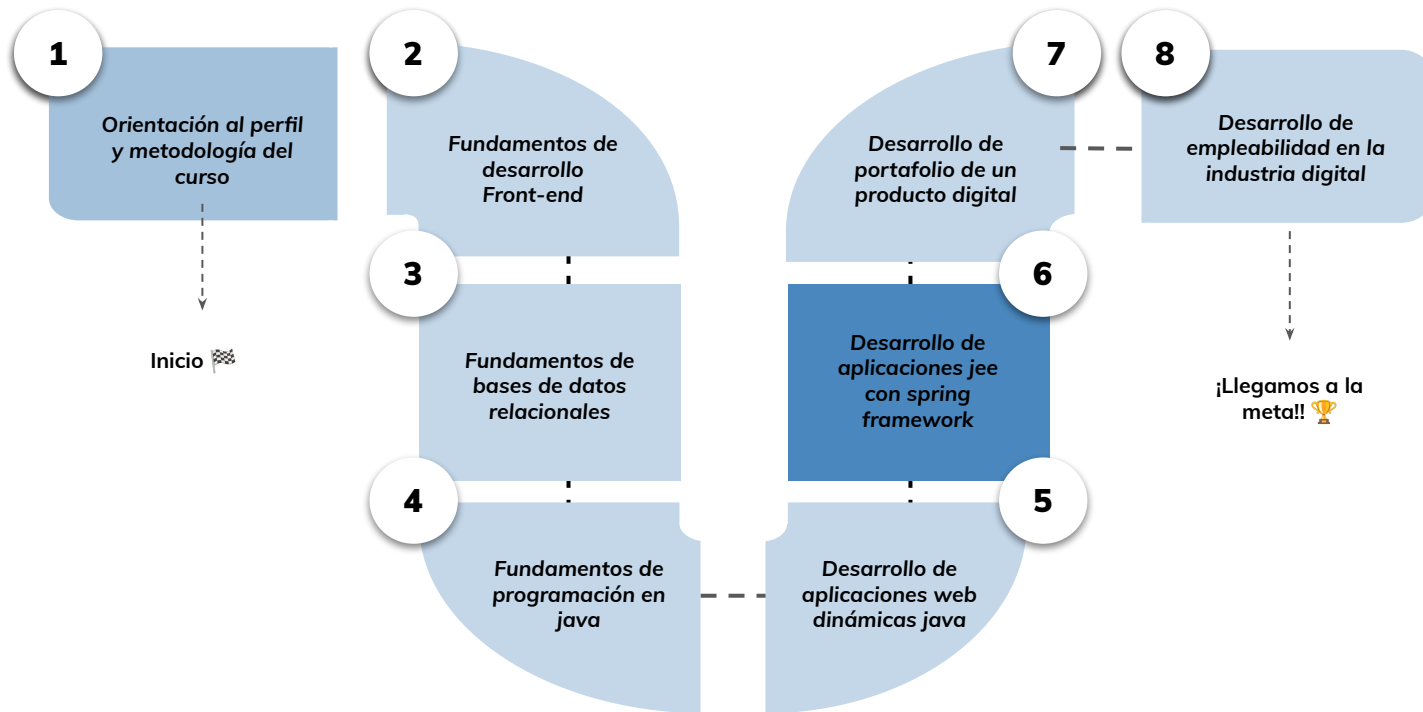


› Acceso a datos en spring framework - Parte I

Aprendizaje Esperado: Implementar capa de acceso a datos en una aplicación web utilizando spring framework para dar solución a un problema.

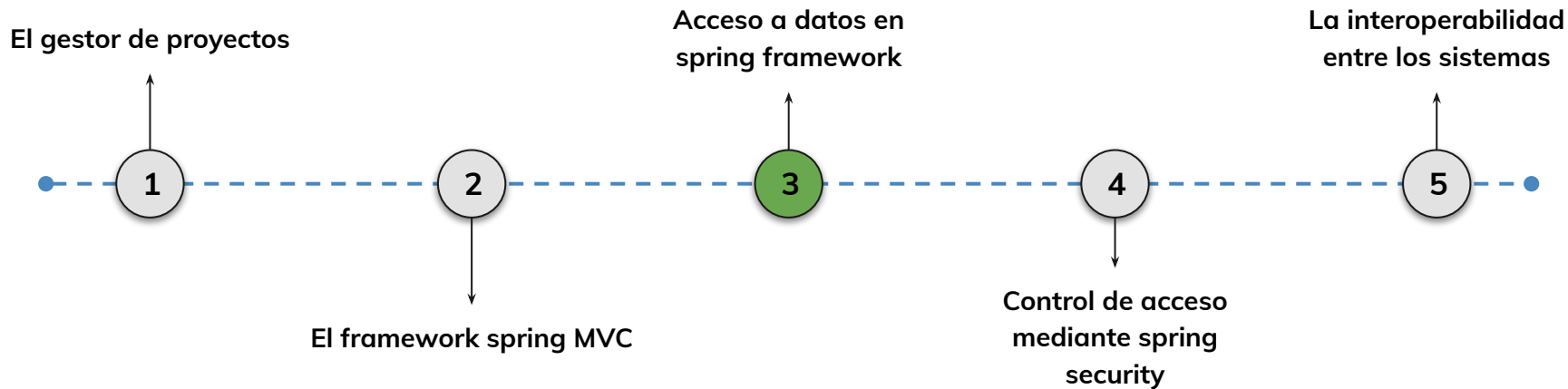
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Implementar capa de acceso a datos en una aplicación web utilizando Spring Framework

Aprenderás a conectar tu aplicación con una base de datos utilizando `JdbcTemplate` y `JPA`, aplicando buenas prácticas de arquitectura y reutilización de código.

Acceso a datos con `JdbcTemplate`

Configuración de `DataSource` y `JdbcTemplate`

DAO con `JdbcTemplate` y operaciones CRUD

Acceso a datos con `JPA`

Clases de Entidad y anotaciones ORM

Uso de `EntityManager` y relaciones entre entidades



Objetivos de aprendizaje

¿Qué aprenderás?



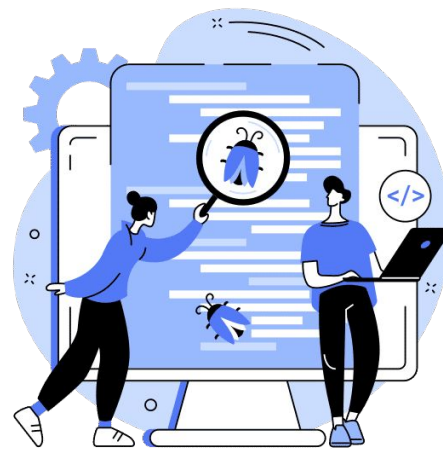
- Aprender a configurar un DataSource en Spring
- Utilizar JdbcTemplate para realizar operaciones CRUD
- Crear clases DAO reutilizables
- Mapear resultados de consultas SQL a objetos Java
- Implementar acceso a datos usando JPA y EntityManager

➤ Acceso a datos mediante JDBC Template

<> JDBC Template

JdbcTemplate es una abstracción de Spring que simplifica la interacción con bases de datos relacionales mediante **JDBC** (Java Database Connectivity). JDBC proporciona una interfaz estándar para interactuar con bases de datos desde Java.

La clase **JdbcTemplate** ejecuta consultas **SQL**, itera sobre el **ResultSet** y recupera los valores llamados, actualiza las instrucciones y las llamadas a procedimientos, "captura" las excepciones y las traduce a las excepciones definidas en el paquete **org.springframework.dao**.



< > JDBC Template

Características Principales de JdbcTemplate:

- **Sencillez y Productividad** : reduce la complejidad de la escritura de código JDBC manual al manejar tareas comunes, como la apertura y cierre de conexiones, el manejo de excepciones y la creación de declaraciones SQL.
- **Manejo de Excepciones Automático** : maneja automáticamente la gestión de excepciones JDBC, convirtiéndolas en excepciones de la jerarquía de Spring `DataAccessException`.
- **Callback y Mapeo de Resultados** : proporciona capacidades de mapeo de resultados que convierten automáticamente los resultados de la consulta en objetos Java.
- **Uso de RowMapper y ResultSetExtractor:** `RowMapper` es una interfaz funcional que permite mapear filas de un `ResultSet` a objetos Java, mientras que `ResultSetExtractor` es una interfaz para extraer información del `ResultSet`.



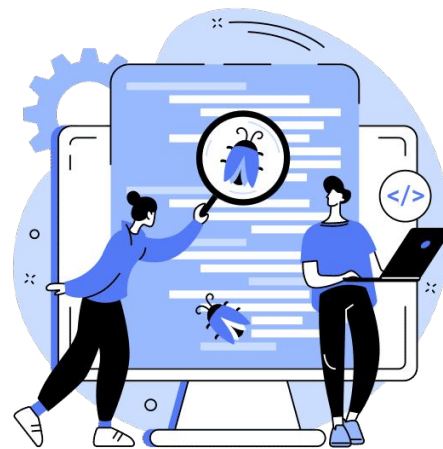
➤ Configurando un DataSource en Spring

< > Configurando un DataSource

Antes de utilizar JdbcTemplate, debes configurar un **DataSource** en tu aplicación Spring.

Un DataSource proporciona la conexión a la base de datos. Puedes configurarlo en el archivo de configuración de Spring, ya sea utilizando el XML o las anotaciones preestablecidas.

Configurar un DataSource en Spring es esencial para permitir el acceso a la base de datos desde tu aplicación.



< > Configurando un DataSource

1. Configuración del Proyecto:

Primero, asegúrate de tener las dependencias necesarias en tu archivo pom.xml:

```
<dependencies>
  <!-- Otras dependencias de Spring -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```



< > Configurando un DataSource

2. Configurar el DataSource en el archivo `application.properties` o `application.yml`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/nombre_de_tu_bd
spring.datasource.username=usuario
spring.datasource.password=contraseña
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```



< > Configurando un DataSource

3. **Configuración XML** : Si estás utilizando una configuración XML en tu aplicación, puedes configurar un DataSource mediante elementos `<bean>` en el archivo de configuración de Spring. Por ejemplo:

```
<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="com.mysql.jdbc.Driver" />
  <property name="url" value="jdbc:mysql://localhost:3306/mydatabase" />
  <property name="username" value="username" />
  <property name="password" value="password" />
</bean>
```



< > Configurando un DataSource

4. **Configuración mediante Anotaciones:** Esta clase de configuración utiliza la anotación `@Configuration` para indicar que es una configuración de Spring y define dos beans: uno para el `DataSource` y otro para el `DataSourceTransactionManager`, que se utiliza para administrar transacciones.

```
@Configuration
public class ConfiguracionBD {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/tu_basedatos");
        dataSource.setUsername("usuario");
        dataSource.setPassword("contraseña");
        return dataSource;
    }

    @Bean
    public DataSourceTransactionManager transactionManager() {
        return new DataSourceTransactionManager(dataSource());
    }
}
```



< > Configurando un DataSource

Ambos métodos son válidos, y la elección entre ellos depende de las preferencias y requisitos de tu aplicación. La configuración mediante anotaciones es más común en aplicaciones Spring modernas debido a su simplicidad y legibilidad. En cualquier caso, el DataSource es un componente clave para establecer la conexión con una base de datos, y configurarlo adecuadamente es esencial para el correcto funcionamiento de tu aplicación.

```
@Configuration
public class ConfiguracionBD {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/tu_basedatos");
        dataSource.setUsername("usuario");
        dataSource.setPassword("contraseña");
        return dataSource;
    }

    @Bean
    public DataSourceTransactionManager transactionManager() {
        return new DataSourceTransactionManager(dataSource());
    }
}
```



➤ Utilizando JDBC Template para acceso a datos

< > JDBC Template para acceso a datos

Paso 1: Configurar el DataSource en tu aplicación

Configura el DataSource en tu aplicación. Puedes hacerlo utilizando anotaciones en una clase de configuración o mediante un archivo de configuración XML como aprendimos anteriormente.

```
@Configuration
public class ConfiguracionBD {

    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/tu_basedatos");
        dataSource.setUsername("usuario");
        dataSource.setPassword("contraseña");
        return dataSource;
    }

    @Bean
    public DataSourceTransactionManager transactionManager() {
        return new DataSourceTransactionManager(dataSource());
    }
}
```

< > JDBC Template para acceso a datos

Paso 2: Crear un Componente que Utilice JdbcTemplate

Puedes inyectar JdbcTemplate en tus componentes y utilizarlo para interactuar con la base de datos. Veamos un ejemplo de un servicio que utiliza JdbcTemplate para realizar operaciones de base de datos:

En este ejemplo, el servicio MiServicio utiliza JdbcTemplate para realizar una consulta que obtiene todos los usuarios de la base de datos y otra operación que inserta un nuevo usuario.

```
@Service
public class MiServicio {

    private final JdbcTemplate jdbcTemplate;

    @Autowired
    public MiServicio(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public List<Usuario> obtenerTodosLosUsuarios() {
        String sql = "SELECT * FROM usuarios";
        return jdbcTemplate.query(sql, (rs, rowNum) ->
            new Usuario(
                rs.getString("id"),
                rs.getString("nombre"),
                rs.getString("email")
            )
        );
    }

    public void insertarUsuario(Usuario usuario) {
        String sql = "INSERT INTO usuarios (nombre, email) VALUES (?, ?)";
        jdbcTemplate.update(sql, usuario.getNombre(), usuario.getEmail());
    }
}
```

< > JDBC Template para acceso a datos

Paso 3: Utilizar el Servicio en un Controlador o en Otros Componentes:

Este controlador expone dos endpoints, uno para obtener todos los usuarios y otro para insertar un nuevo usuario.

```
@RequestMapping("/usuarios")
public class MiControlador {
    @Autowired
    private final MiServicio miServicio;

    public MiControlador(MiServicio miServicio) {
        this.miServicio = miServicio;
    }

    @GetMapping
    public List<Usuario> obtenerTodosLosUsuarios() {
        return miServicio.obtenerTodosLosUsuarios();
    }

    @PostMapping
    public void insertarUsuario(@RequestBody Usuario usuario) {
        miServicio.insertarUsuario(usuario);
    }
}
```



LIVE CODING

Completando la Wallet:

- Es hora de completar nuestro proyecto de billetera virtual. Para esto, vamos a agregar la clase `Cuenta` que contendrá la información de una cuenta bancaria: `id:numeroCuenta`, `saldoActual`, `tipoMoneda`.
- Además, vamos a crear la clase de `Servicio` para `Cuenta` que nos permitirá realizar operaciones como: `consultarSaldo`, `cambiarMoneda`, `extraerDinero`, `depositarDinero`, etc.
- Luego, crearemos una clase de **configuración** para poder definir el `datasource`. Recuerda crear la base de datos con anterioridad.

Tiempo: 30 minutos

➤ DAO con JDBC Template

< > DAO con JDBC Template



Crear un **DAO** (Data Access Object) implica **interactuar con una base de datos** .
Al integrar **Spring** y su módulo **JdbcTemplate** , los desarrolladores pueden aprovechar una poderosa combinación para simplificar y eficientizar la interacción con bases de datos relacionales en aplicaciones Java.



< > DAO con JDBC Template

El DAO actúa como una capa intermedia entre la aplicación y la base de datos. Proporciona una interfaz de alto nivel para acceder y manipular datos, abstrayendo los detalles específicos de la base de datos. La integración de JdbcTemplate con Spring facilita la gestión de la conexión, las transacciones y la ejecución de consultas SQL, eliminando la necesidad de manejar manualmente tareas repetitivas y propensas a errores.



< > DAO con JDBC Template

JdbcTemplate es una clase proporcionada por el módulo JDBC de Spring que simplifica la ejecución de operaciones de base de datos utilizando JDBC puro. Elimina la necesidad de manejar manualmente la creación y liberación de recursos de conexión, gestionando estos aspectos de manera transparente. Además, simplifica la escritura de código al proporcionar métodos convenientes para ejecutar consultas y actualizar datos.



< > DAO con JDBC Template

Configuración de la Base de Datos:

En una aplicación Spring con JdbcTemplate, la configuración de la base de datos se realiza generalmente en un archivo de configuración, ya sea mediante anotaciones o mediante un archivo XML. La configuración incluye detalles como la fuente de datos (**DataSource**), la cual puede ser configurada para utilizar un pool de conexiones, y el objeto JdbcTemplate que se basará en esta fuente de datos.



< > DAO con JDBC Template

Configuración de la Base de Datos

En nuestro caso utilizaremos la base de datos MySQL, por lo cual agregaremos la dependencia necesaria en el archivo pom.xml

```
<!-- Dependencia MySQL JDBC Driver -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.23</version> <!-- Ajusta la versión según tu entorno -->
</dependency>
```



< > DAO con JDBC Template

Luego, debemos configurar el DataSource en el archivo `application.properties` para MySQL.

```
# Configuración de la base de datos MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/nombre_de_tu_base_de_datos
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
spring.datasource.username=tu_usuario
spring.datasource.password=tu_contraseña
spring.jpa.hibernate.ddl-auto=update
```



< > DAO con JDBC Template

Los datos del application.properties deben coincidir con la configuración de la clase anotada con @Configuration.

```
@Configuration
public class ConfiguracionBD {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/tu_basedatos");
        dataSource.setUsername("usuario");
        dataSource.setPassword("contraseña");
        return dataSource;
    }
}
```



< > DAO con JDBC Template

Finalmente creamos una clase DAO para manipular la conexión y consultas a la base de datos. En este ejemplo creamos la clase UsuarioDAO con el atributo jdbcTemplate.

```
public class UsuarioDao {  
  
    @Autowired  
    private final JdbcTemplate jdbcTemplate;  
  
    public UsuarioDao(JdbcTemplate jdbcTemplate) {  
        this.jdbcTemplate = jdbcTemplate;  
    }  
}
```



➤ Creando Querys con parámetros

< > Creando Querys que reciben parámetros

Realizar consultas que involucran parámetros es una parte esencial de cualquier aplicación de acceso a datos. Con JdbcTemplate, la creación y ejecución de consultas parametrizadas es sencilla y segura. Veamos un ejemplo de cómo realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) utilizando JdbcTemplate.



< > Creando Querys que reciben parámetros

1. Crear Registros con Parámetros:

Para la operación de creación, supongamos que queremos agregar un método a nuestro DAO para insertar un nuevo usuario en la tabla. Vamos a utilizar un método que acepta un objeto Usuario como parámetro.

```
public void insertarUsuario(Usuario usuario) {  
    String sql = "INSERT INTO usuario (id, nombre, email) VALUES (?, ?, ?)";  
    jdbcTemplate.update(sql, usuario.getId(), usuario.getNombre(), usuario.getEmail());  
}
```

Aquí, el método **insertarUsuario** utiliza el método **update** de JdbcTemplate, que acepta la consulta SQL y los valores de los parámetros.



< > Creando Querys que reciben parámetros

2. Actualizar Registros con Parámetros:

Si queremos permitir la actualización de la información del usuario, podríamos implementar un método como el siguiente:

```
public void actualizarUsuario(Usuario usuario) {  
    String sql = "UPDATE usuario SET nombre = ?, email = ? WHERE id = ?";  
    jdbcTemplate.update(sql, usuario.getNombre(), usuario.getEmail(), usuario.getId());  
}
```

Este método utiliza una consulta **UPDATE** parametrizada para **actualizar** la información de un usuario específico.



< > Creando Querys que reciben parámetros

3- Eliminar Registros con Parámetros:

Finalmente, para la operación de eliminación, podríamos tener un método que elimine un usuario según su ID.

```
public void eliminarUsuarioPorId(String id) {  
    String sql = "DELETE FROM usuario WHERE id = ?";  
    jdbcTemplate.update(sql, id);  
}
```

Este método utiliza una consulta **DELETE** parametrizada para **eliminar** un usuario **según su ID**.



➤ Mapeando los resultados de una consulta a objetos

< > Mapeando los resultados de una consulta a objetos

Mapear los resultados de una consulta a objetos es una parte clave del proceso de acceso a datos en cualquier aplicación. En el contexto de JdbcTemplate en Spring, este proceso se simplifica mediante el uso de **RowMappers**.

En JdbcTemplate, un **RowMapper** es una interfaz funcional que define el método **mapRow**, responsable de mapear un conjunto de resultados (un registro de la base de datos) a un objeto Java.



< > Mapeando los resultados de una consulta a objetos

Veamos un ejemplo de una clase exclusiva para el mapeo de objetos. En esta implementación, creamos una clase separada llamada **UsuarioRowMapper** que **implementa la interfaz RowMapper**. La interfaz RowMapper requiere la implementación del método **mapRow**, que realiza el mapeo de la fila del conjunto de resultados a un objeto Usuario.

```
public class UsuarioRowMapper implements RowMapper<Usuario> {  
  
    @Override  
    public Usuario mapRow(ResultSet resultSet, int rowNum) throws SQLException {  
        Usuario usuario = new Usuario();  
        usuario.setId(resultSet.getString("id"));  
        usuario.setNombre(resultSet.getString("nombre"));  
        usuario.setEmail(resultSet.getString("email"));  
        return usuario;  
    }  
}
```



< > Mapeando los resultados de una consulta a objetos

Finalmente, en el DAO, utilizamos la instancia de **UsuarioRowMapper** como **argumento en el método query** . Este enfoque resulta en un código más explícito y es fácil de entender para aquellos que están comenzando con Java y Spring. Además, es útil si deseas realizar operaciones más avanzadas durante el mapeo.

```
public List<Usuario> buscarUsuariosPorNombre(String nombre) {  
    String sql = "SELECT * FROM usuario WHERE nombre = ?";  
    return jdbcTemplate.query(sql, new Object[]{nombre}, new UsuarioRowMapper());  
}
```



< > Mapeando los resultados de una consulta a objetos

También es posible realizar el mapeo de resultados **sin una clase RowMapper separada**, especialmente para aplicaciones más simples. Puedes utilizar una instancia anónima de RowMapper directamente en el método query.

```
public List<Usuario> buscarUsuariosPorNombre(String nombre) {  
    String sql = "SELECT * FROM usuario WHERE nombre = ?";  
  
    return jdbcTemplate.query(sql, new Object[]{nombre}, new RowMapper<Usuario>() {  
        @Override  
        public Usuario mapRow(ResultSet resultSet, int rowNum) throws SQLException {  
            Usuario usuario = new Usuario();  
            usuario.setId(resultSet.getString("id"));  
            usuario.setNombre(resultSet.getString("nombre"));  
            usuario.setEmail(resultSet.getString("email"));  
            return usuario;  
        }  
    });  
}
```



< > **Mapeando los resultados de una consulta a objetos**

En este ejemplo, hemos creado una instancia anónima de `RowMapper` directamente en el método `query`. La implementación de `RowMapper` está contenida dentro de la declaración de la instancia anónima.

Este enfoque es una alternativa válida cuando se desea mantener una estructura más tradicional. Ambos enfoques (usar clases separadas o instancias anónimas) son aceptados en Java y pueden adaptarse según las preferencias de codificación del equipo y la complejidad del código.



< > Modificando datos

Como ya hemos visto, si queremos modificar datos con JdbcTemplate en Spring se realiza comúnmente utilizando el método **update**. Este método permite ejecutar consultas SQL que realizan operaciones de modificación en la base de datos, como **INSERT**, **UPDATE** y **DELETE**.

En el caso de querer modificar solo un atributo del registro, podemos recibirlo a través de los parámetros. Por ejemplo, este método actualiza el campo email de un usuario identificado por su ID con el nuevo email proporcionado.

```
public void actualizarEmail(String id, String nuevoEmail) {  
    String sql = "UPDATE usuario SET email = ? WHERE id = ?";  
    jdbcTemplate.update(sql, nuevoEmail, id);  
}
```



➤ Invocando un DAO desde el servicio

< > Invocando un DAO desde el Servicio

Invocar un DAO desde un servicio es una práctica común en la arquitectura de aplicaciones en Spring. El servicio actúa como una capa intermedia que encapsula la lógica de negocio, y el DAO se encarga de las operaciones de acceso a datos.

Supongamos que tienes un DAO para la entidad Usuario como se mencionó en ejemplos anteriores. Para poder utilizarlo, es necesario crear una instancia única del DAO en la clase de servicio para así acceder a todos sus métodos,

```
@Service
public class UsuarioService {

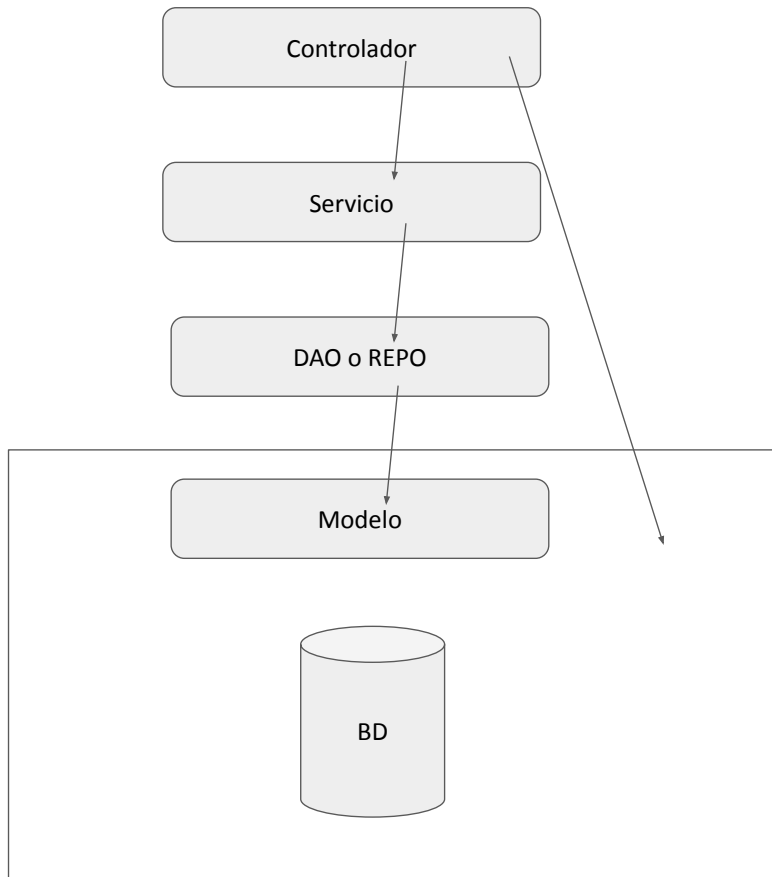
    private final UsuarioDao usuarioDao;

    @Autowired
    public UsuarioService(UsuarioDao usuarioDao) {
        this.usuarioDao = usuarioDao;
    }

    public List<Usuario> obtenerTodosLosUsuarios() {
        return usuarioDao.obtenerTodosLosUsuarios();
    }

    public void guardarUsuario(Usuario usuario) {
        usuarioDao.guardarUsuario(usuario);
    }
}
```





LIVE CODING

Completando la Wallet:

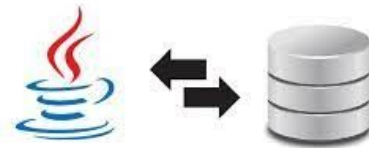
- *Es hora de completar nuestro proyecto de billetera virtual. Para esto, vamos a agregar la clase CuentaDAO y UsuarioDAO.*
- *Además, vamos a crear en los DAO los métodos necesarios para el CRUD de cada correspondiente entidad.*

Tiempo: 30 minutos

➤ Acceso a datos con JPA

< > Acceso a datos con JPA

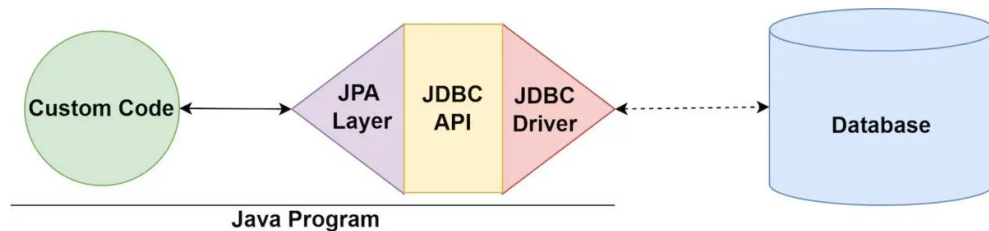
JPA (**J**ava **P**ersistence **A**PI) es la propuesta estándar que ofrece Java para implementar un Framework Object Relational Mapping (**ORM**), que **permite interactuar con la base de datos por medio de objetos**. JPA es el encargado de convertir los objetos Java en instrucciones para el Manejador de Base de Datos (DBMS).



JAVA PERSISTENCE API



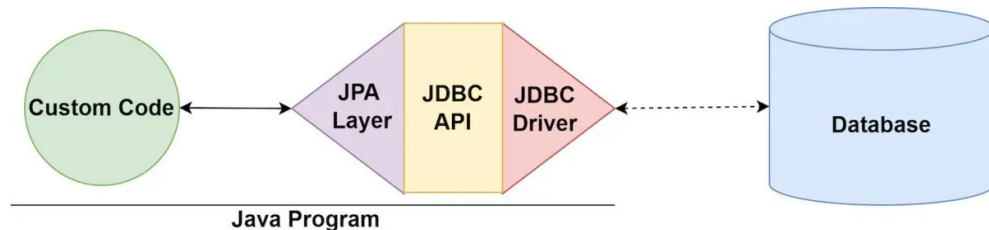
< > Acceso a datos con JPA



JPA crea un estándar para la importante tarea de la correlación relacional de objetos mediante la utilización de **anotaciones** o **XML** para relacionar objetos con una o más tablas de una base de datos.



< > Acceso a datos con JPA



El objetivo que persigue el diseño de esta API es no perder las ventajas de la orientación a objetos al interactuar con una base de datos (siguiendo el patrón de mapeo objeto-relacional), como sí pasaba con EJB2, y permitir usar objetos regulares (conocidos como POJO).



< > Acceso a datos con JPA

Objetivos de JPA:

- **Abstracción de la Capa de Datos** : Esto permite a los desarrolladores trabajar con objetos Java en lugar de consultas SQL, facilitando el desarrollo y mantenimiento del código.
- **Portabilidad** : Los desarrolladores pueden escribir código independiente de la base de datos, y JPA se encargará de traducir las operaciones a la sintaxis específica de cada proveedor.
- **Mapeo Objeto-Relacional (ORM)** : JPA facilita la tarea de mapear objetos Java a tablas de bases de datos y viceversa. Esto reduce la complejidad de las consultas SQL y mejora la legibilidad del código.
- **Transacciones** : JPA gestiona automáticamente las transacciones, asegurando la consistencia de los datos y proporcionando un mecanismo robusto para el control de concurrencia.



< > Acceso a datos con JPA

Características principales:

- **Anotaciones** : JPA utiliza anotaciones Java para mapear entidades a tablas y definir relaciones entre ellas. Esto simplifica el proceso de configuración y evita la necesidad de archivos de configuración XML.
- **Entity Manager** : Permite realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en las entidades y administrar el ciclo de vida de los objetos.
- **Consulta JPQL** : Java Persistence Query Language (JPQL) es un lenguaje de consulta orientado a objetos que permite realizar consultas en entidades Java en lugar de tablas de base de datos.
- **Relaciones entre Entidades** : JPA admite la definición de relaciones entre entidades, como uno a uno, uno a muchos y muchos a muchos. Esto se logra mediante el uso de anotaciones como @OneToOne, @OneToMany, y @ManyToMany.



Acceso a datos con JPA

✓ Ventajas:

- **Portabilidad** : La capacidad de cambiar fácilmente entre diferentes proveedores de bases de datos sin cambiar el código fuente es una ventaja significativa.
- **Mapeo Objeto-Relacional** : simplifica el mapeo de objetos a tablas de bases de datos, eliminando gran parte del código JDBC manual.
- **Consulta JPQL** : La capacidad de realizar consultas utilizando JPQL permite a los desarrolladores trabajar con entidades Java en lugar de preocuparse por la estructura de la base de datos, lo que facilita el desarrollo y mantenimiento del código.

⊖ Desventajas:

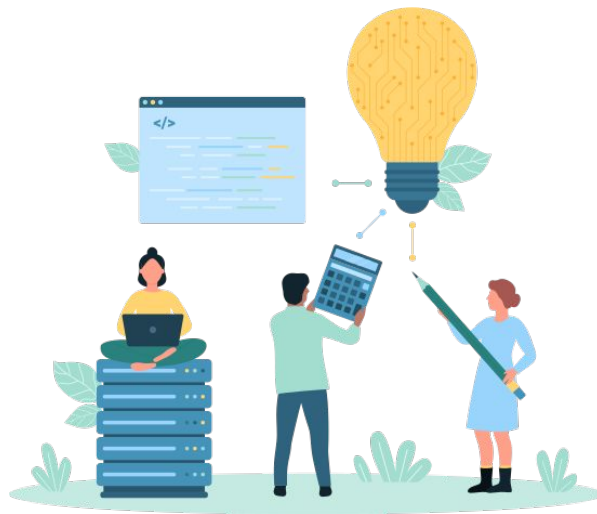
- **Complejidad**: La configuración y el entendimiento completo de JPA pueden requerir tiempo y esfuerzo.
- **Rendimiento**: En comparación con consultas SQL personalizadas, las consultas JPQL pueden tener un rendimiento ligeramente inferior en ciertos casos.
- **Sobreabstracción**: En algunos casos, la abstracción proporcionada por JPA puede resultar excesiva, especialmente para aplicaciones simples.

➤ Clases de Entidad



Clases de Entidad

Las clases de entidad son componentes fundamentales que representan objetos persistentes almacenados en una base de datos relacional. Estas clases sirven como puente entre el mundo orientado a objetos de Java y el mundo de las bases de datos relacionales, facilitando el mapeo de datos entre ambas tecnologías.





Clases de Entidad



En otras palabras, los objetos de entidades pueden ser clases concretas o clases abstractas. Podemos decir que **cada Entidad corresponderá con una tabla de nuestra Base de Datos**.

Las clases entidad se identifican con la anotación `@Entity` (a nivel clase).

```
@Entity ←  
public class Usuario {  
    // Atributos, constructores, métodos, etc.  
}
```




Clases de Entidad

Definición de Clase de Entidad:

Una clase de entidad en JPA es una clase de Java ordinaria que se anota con la anotación **@Entity**. **Esta anotación indica a JPA que la clase debe ser persistente**, es decir, que sus instancias pueden ser almacenadas y recuperadas de una base de datos.

```
@Entity ←  
public class Usuario {  
    // Atributos, constructores, métodos, etc.  
}
```



Clases de Entidad



Definición de Clase de Entidad:

Una clase de entidad en JPA es una clase de Java ordinaria que se anota con la anotación **@Entity**. **Esta anotación indica a JPA que la clase debe ser persistente**, es decir, que sus instancias pueden ser almacenadas y recuperadas de una base de datos.

```
3 import javax.persistence.Entity;
4
5 @Entity
6 public class Usuario {
7     String id;
8     String nombre;
9     String email;
10 }
```



Clases de Entidad



Identidad y Clave Primaria:

Cada entidad en JPA **debe tener una clave primaria única** que la distinga de otras instancias de la misma entidad. La anotación `@Id` se utiliza para marcar el campo que actuará como clave primaria.

```
6 @Entity
7 public class Usuario {
8
9     @Id
10    String id;
11    String nombre;
12    String email;
13
```

En este ejemplo, el campo **id** se define como **la clave primaria de la entidad Usuario** .



Clases de Entidad



Ventajas de las Entity class:

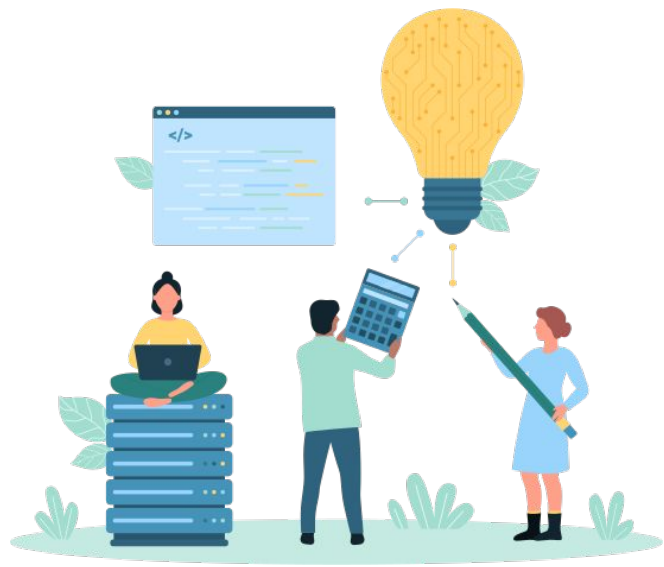
- **Reutilización de Código** : Al representar objetos persistentes como clases de entidad, se promueve la reutilización de código y la organización lógica de la aplicación.
- **Facilita el Mapeo Objeto-Relacional:** JPA simplifica el mapeo de objetos a tablas de bases de datos a través de las clases de entidad, abstrayendo gran parte de la complejidad asociada con las operaciones de persistencia.
- **Soporte para Relaciones Complejas** : Las clases de entidad permiten modelar relaciones complejas entre objetos, facilitando el diseño de bases de datos relacionales más sofisticadas.
- **Integración con el Ciclo de Vida:** JPA proporciona mecanismos para gestionar el ciclo de vida de las entidades, permitiendo ejecutar lógica personalizada en momentos específicos, como antes o después de persistir una entidad.

➤ Mapeo con anotaciones

< > Mapeo con anotaciones

Como ya hemos visto anteriormente, las bases de datos relacionales almacenan la información mediante tablas, filas, y columnas, de manera que para almacenar un objeto en la base de datos, hay que realizar una correlación entre el sistema orientado a objetos de Java y el sistema relacional de nuestra base de datos.

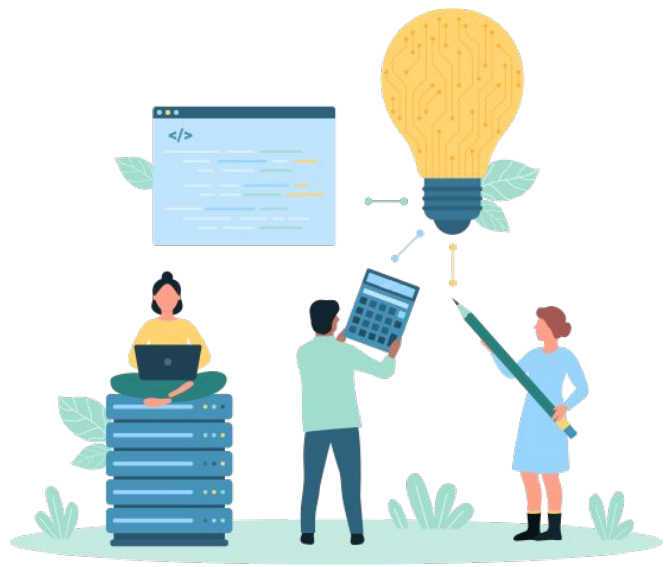
JPA nos permite realizar dicha correlación de forma sencilla, realizando por sí misma toda la conversión entre nuestros objetos y las tablas de una base de datos.



< > Mapeo con anotaciones

Esta conversión se denomina ORM (Object Relational Mapping - Mapeo Relacional de Objetos), y puede configurarse a través de metadatos (anotaciones).

Las anotaciones nos permiten configurar el mapeo de una entidad dentro del mismo archivo donde se declara la clase, de este modo, relaciona las clases contra las tablas y los atributos contra las columnas. Mediante las anotaciones vamos a explicarle al ORM como debe transformar la entidad en una tabla de base de datos.



< > Mapeo con anotaciones

Algunas de las principales anotaciones son:

- **@Entity**: Declara la clase como una Entidad.
- **@Table**: Declara el nombre de la Tabla con la que se mapea la Entidad.
- **@Id**: Declara un atributo como la clave primaria de la Tabla.
- **@GeneratedValue**: **Declara la manera en la que el atributo clave primaria va a ser inicializado** . Esto puede ser de forma manual, automática o a partir de una secuencia determinada. La estrategia **IDENTITY** es la más simple de usar, ya que JPA asume que ésta columna es auto generada. Esto provoca que el contador de la columna incremente en 1 cada vez que un nuevo objeto es insertado.



< > Mapeo con anotaciones

- **@Column**: Declara que un atributo se mapea con una columna de la tabla.
- **@Enumerated**: Declara que un atributo es de alguno de los valores definidos en un Enum (lista de valores constantes). Los valores de un tipo enumerado tienen asociado implícitamente un tipo ordinal.
- **@Temporal**: Declara que se está tratando de un atributo que va a trabajar con fechas, entre paréntesis, debemos especificarle qué estilo de fecha va a manejar en la base de datos:

@Temporal(TemporalType.DATE)

@Temporal(TemporalType.TIME)

@Temporal(TemporalType.TIMESTAMP)



< > Mapeo con anotaciones

Por ejemplo, para la generación automática de un ID numérico incremental, podríamos utilizar la anotación `@GeneratedValue` de la siguiente manera;

```
8 @Entity
9 public class Cuenta {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     Long id;
14 }
```

La estrategia `IDENTITY` es la que le da el comportamiento incremental.



< > Mapeo con anotaciones

En cambio, si quisieramos que se genere un id compuesto por una cadena alfanumerica única, utilizamos la anotación `@GeneratedValue` con el generador “`uuid`”

```
7 @Entity
8 public class Usuario {
9
10     @Id
11     @GeneratedValue(generator = "uuid")
12     String id;
```

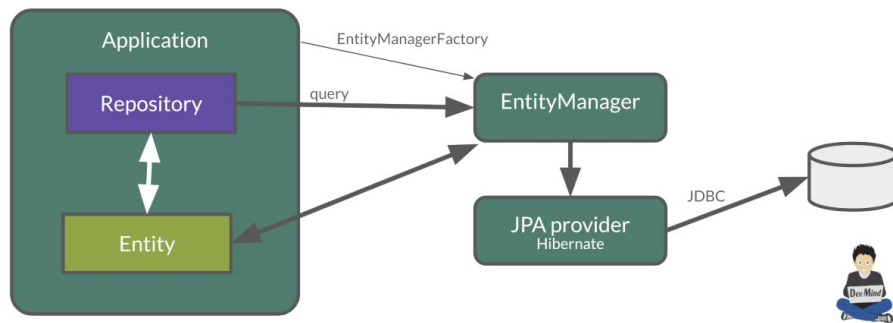


➤ El EntityManager en JPA

El EntityManager en JPA

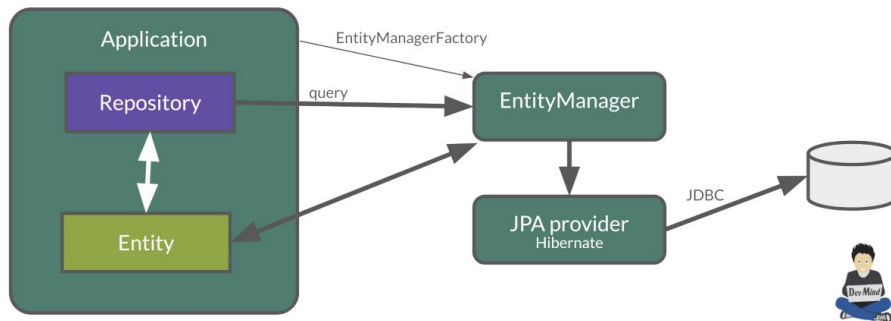
El Entity Manager es un componente central en Java Persistence API (JPA), y su función principal es gestionar el ciclo de vida de las entidades persistentes en una aplicación Java.

Proporciona una interfaz para interactuar con el contexto de persistencia y realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en las entidades mapeadas a una base de datos relacional.



El EntityManager en JPA

El Entity Manager es una interfaz de programación de aplicaciones (API) que forma parte de JPA y se utiliza para gestionar las operaciones de persistencia en entidades. Puede ser considerado como una especie de "puente" entre el mundo de objetos de Java y la base de datos relacional.



El **EntityManager** se obtiene a partir de un **EntityManagerFactory**, que es responsable de crear instancias de Entity Manager.

El EntityManager en JPA

Componentes :

- **Persistence Context** : Este contexto mantiene un conjunto de entidades gestionadas, lo que significa que el Entity Manager está al tanto de los cambios realizados en esas entidades durante la transacción. El Persistence Context asegura la coherencia y la sincronización de los objetos de entidad.

El EntityManager en JPA

- **Cache de Primer Nivel:** Dentro del Persistence Context, el Entity Manager mantiene una caché de primer nivel que almacena las entidades que han sido recuperadas o gestionadas durante la transacción. Esto mejora el rendimiento al evitar múltiples recuperaciones de la misma entidad.
- **Transacciones :** El Entity Manager se integra estrechamente con el sistema de transacciones de Java, y sus operaciones están típicamente encapsuladas dentro de una transacción. Las transacciones garantizan la consistencia y la atomicidad de las operaciones realizadas en la base de datos.

LIVE CODING

Completando la Wallet:

Ahora vamos a utilizar las anotaciones de generación automática (**@GeneratedValue**) para los ID de las clases entidad:

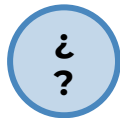
- El **Id** de Usuario tendrá una configuración de generación automática “uuid”.
- El **Id** de Cuenta tendrá una configuración de generación automática de tipo Identity.

Tiempo: 10 minutos

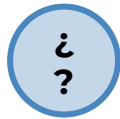
#Momentode Preguntas...



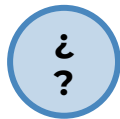
¿Qué ventajas ofrece *JdbcTemplate* frente al *JDBC* tradicional?



¿Cuándo conviene usar *JPA* en lugar de *JdbcTemplate*?



¿Qué función cumple el *RowMapper* en *Spring*?



¿Qué diferencia hay entre `@GeneratedValue(strategy = IDENTITY)` y `uuid`?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

DAO con JdbcTemplate

DAO con JdbcTemplate

Contexto: 🙌

Estás desarrollando una aplicación de billetera virtual. Hasta ahora has implementado el registro y autenticación de usuarios. El equipo ahora necesita agregar persistencia de datos para cuentas y usuarios, siguiendo una arquitectura limpia.

Consigna: 📝

Implementa la clase UsuarioDAO y CuentaDAO utilizando JdbcTemplate en Spring. Deberás incluir operaciones básicas: listar, insertar, actualizar y eliminar. Luego, crea un servicio que consuma los métodos del DAO.

Tiempo 🕒: 30 minutos

Paso a paso: ⚙️

1. Configura el DataSource en application.properties.
2. Crea la clase de configuración que devuelve el JdbcTemplate.
3. Implementa UsuarioDAO con métodos como listarUsuarios(), insertarUsuario(), etc.
4. Crea un servicio UsuarioService que invoque el DAO.
5. Prueba la funcionalidad con un controlador REST.

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Comprendimos cómo configurar el acceso a base de datos con Spring**
- ✓ **Creamos un DAO utilizando JdbcTemplate**
- ✓ **Usamos el patrón RowMapper para mapear objetos**
- ✓ **Conectamos la capa DAO con el servicio**
- ✓ **Exploramos la diferencia entre JdbcTemplate y JPA**
- ✓ **Mapeamos entidades con anotaciones**
- ✓ **Utilizamos EntityManager para persistencia**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ———— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

